

Web Application Security Testing Report

Prepared by - Mubasshera Zaidy

Objective

The objective of this project was to perform a comprehensive Vulnerability Assessment and Penetration Testing (VAPT) on the OWASP Juice Shop web application. The assessment aimed to identify security flaws, demonstrate exploitation through controlled testing, and provide actionable recommendations for remediation.

Using industry-standard tools such as OWASP ZAP, Burp Suite, and manual testing techniques, several vulnerabilities were discovered, including DOM-based Cross-Site Scripting (XSS), SQL injection, broken authentication, insecure direct object references (IDOR), and missing security headers.

The findings highlight that Juice Shop is intentionally insecure and serves as a training ground for OWASP Top 10 vulnerabilities. The overall risk rating of the application is High, with multiple critical issues that could lead to data theft, account compromise, and unauthorized access if exploited in a real-world scenario.

Environment Setup

- Operating System: Kali Linux VM.
- Target Application: OWASP Juice Shop (Docker container).
- Command used:

```
sudo docker run --rm -p 3000:3000 bkimminich/juice-shop
```

- Accessed via : <http://localhost:3000>

Tools used for Vulnerability Assessment:

- OWASP ZAP → Automated scanning, spidering, alerts.
- Burp Suite → Manual interception and payload injection.
- Browser (Firefox/Chromium) → Proxy configured to ZAP (127.0.0.1:8080).

Methodology

The Vulnerability Assessment and Penetration Testing (VAPT) was conducted using a black-box approach, simulating an external attacker with limited knowledge of the application. The following steps were followed systematically:

1. Planning and Scope

- Defined the target application: *OWASP Juice Shop* running on `http://localhost:3000`
- Established objectives: identify, exploit, and document vulnerabilities aligned with the OWASP Top 10.
- Tools selected: OWASP ZAP, Burp Suite, browser DevTools, and manual testing techniques.

2. Information Gathering

- Collected details about the application's structure, endpoints, and technologies used.
- Observed client-side behavior using browser DevTools.
- Intercepted traffic through ZAP/Burp to identify API calls and parameters.

4. Penetration Testing

- Exploited identified vulnerabilities to confirm their impact:

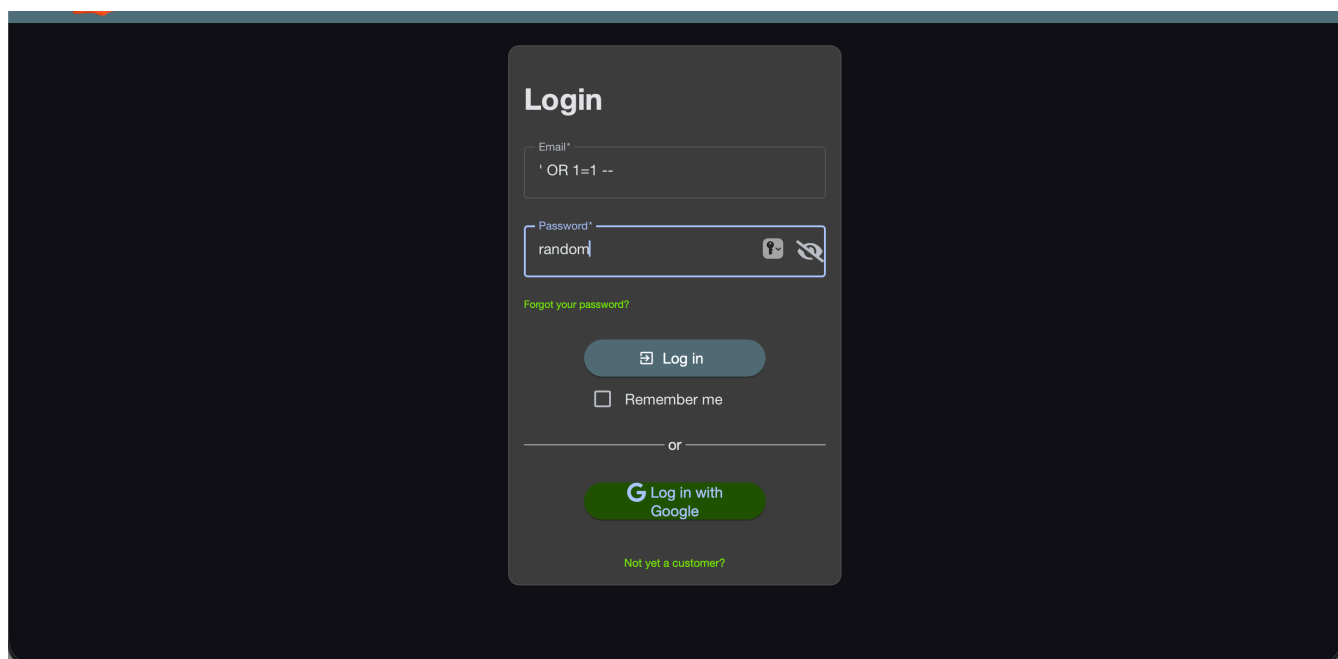
1. SQL Injection – Admin Account Compromise

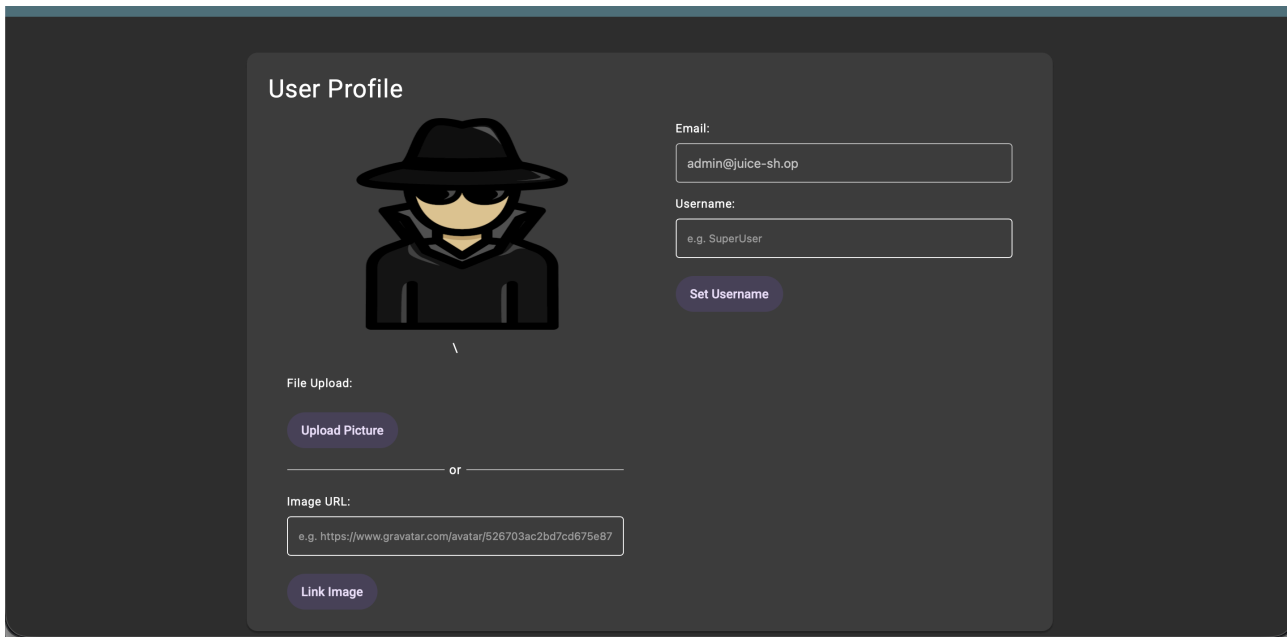
Title: SQL Injection in Login Form – Direct Admin Account Access

Description: The login form is vulnerable to SQL Injection. By injecting crafted input into the username field, the application executed unintended SQL queries. This allowed direct login into the **administrator account**, granting full control over the application.

Evidence:

- Payload used: ' OR 1=1 --
- Screenshot of the login request with the injected payload.
- Screenshot showing successful login into the **Admin account dashboard**.





The image shows a 'User Profile' form on a dark background. At the top left is a hacker icon (a yellow face with a black hat and sunglasses). To the right of the icon are two input fields: 'Email:' with the value 'admin@juice-sh.op' and 'Username:' with the value 'e.g. SuperUser'. Below the 'Username:' field is a 'Set Username' button. Below the hacker icon is a 'File Upload:' section with an 'Upload Picture' button. Below that is an 'Image URL:' section with an input field containing the example URL 'e.g. https://www.gravatar.com/avatar/526703ac2bd7cd675e87' and a 'Link Image' button. The form is titled 'User Profile' at the top left.

Impact:

- Complete compromise of the application.
- Unauthorized access to administrative functions.
- Ability to manage users, view sensitive data, and escalate attacks further.

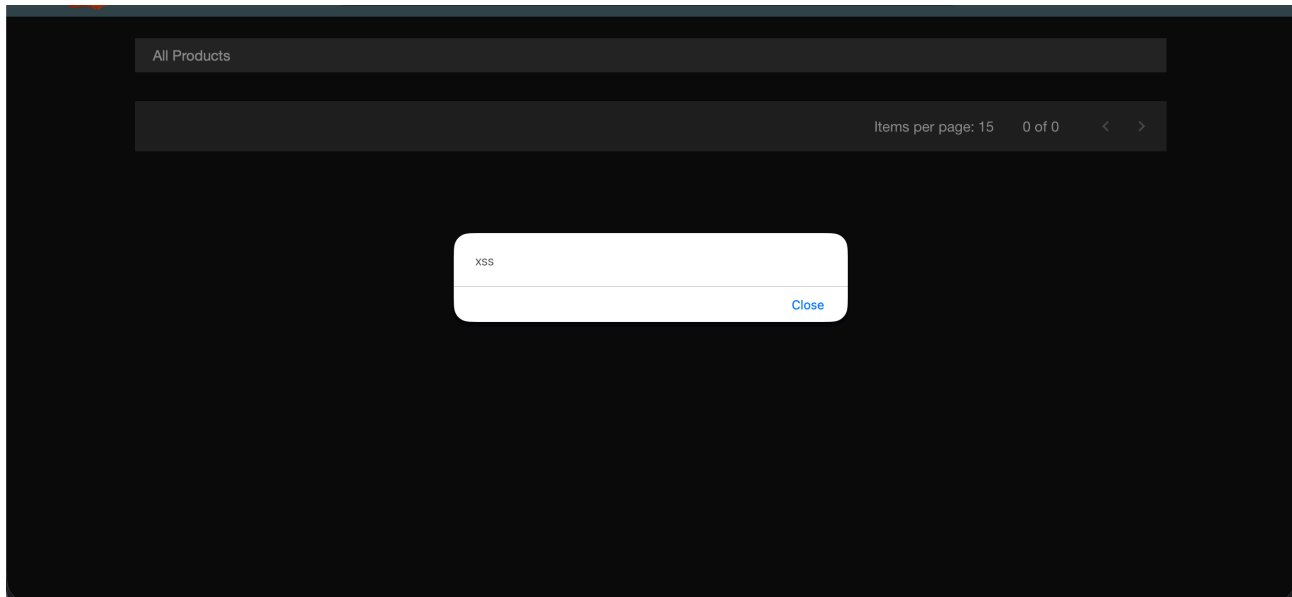
Severity: Critical

Recommendation:

- Use parameterized queries (prepared statements).
- Implement strict input validation and sanitization.
- Restrict access to the admin panel with multi-factor authentication.
- Regularly test authentication endpoints against SQLi attacks.

2. Reflected Cross-Site Scripting (XSS)

- **Description:** The feedback form reflects unsanitized input back to the user, enabling script execution.
- **Evidence:** `<script>alert('xss')</script>` reflected in response and executed in browser.



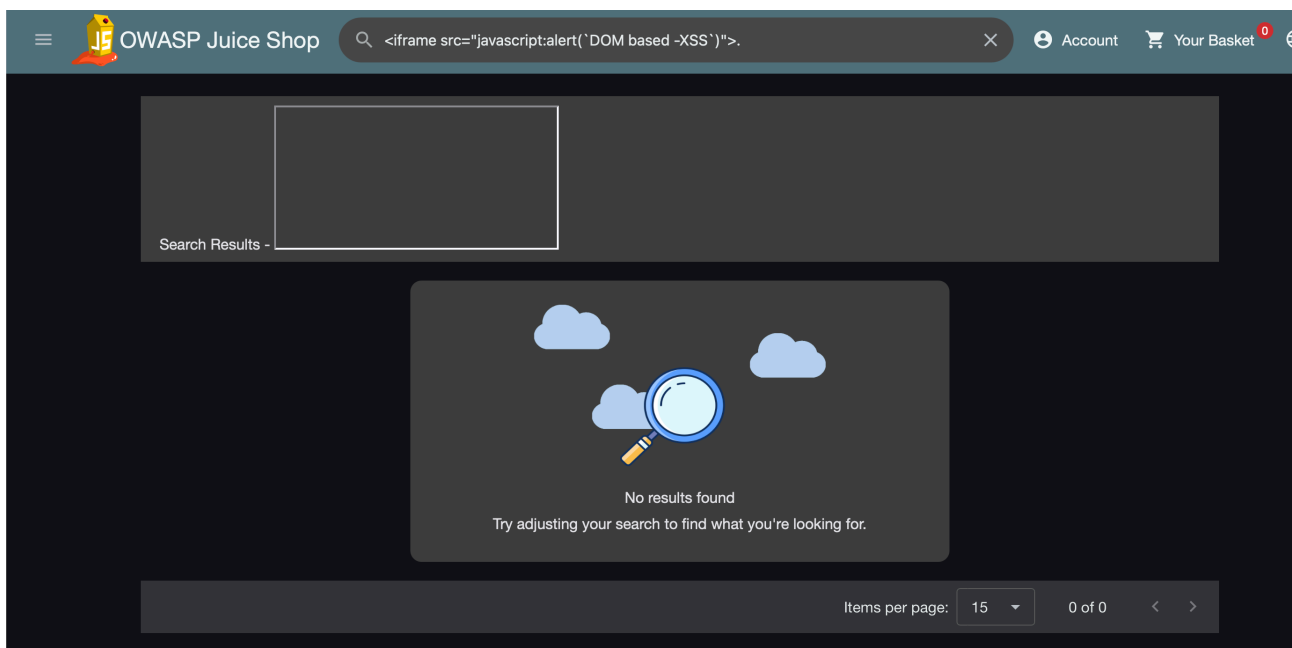
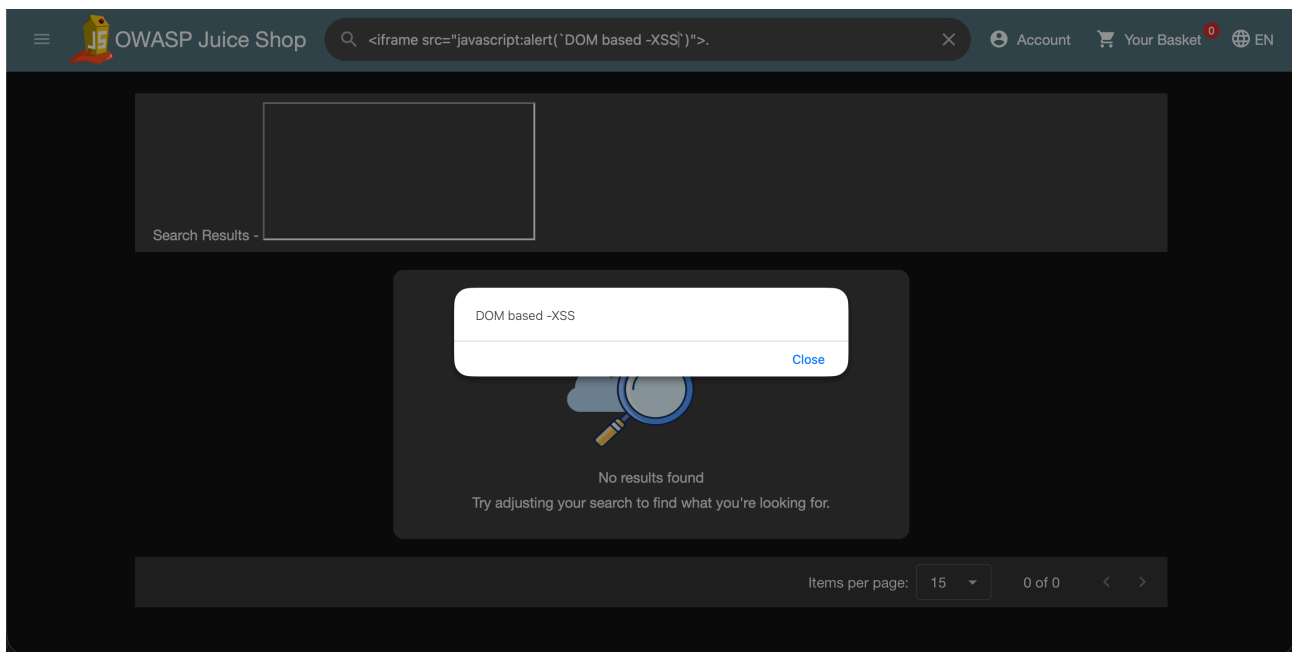
- **Impact:** Cookie theft, phishing attacks.
- **Severity:** High
- **Recommendation:** Validate and encode user input, enforce CSP.

3. DOM-Based Cross-Site Scripting (XSS)

Description: The application's client-side JavaScript directly writes unsanitized user input into the DOM. This allows attackers to inject malicious scripts that execute in the victim's browser. Unlike reflected XSS, DOM-based XSS occurs entirely on the client side without server involvement.

Evidence:

- Payload used: `<iframe src="javascript:alert(`DOM based -XSS`)">`.



Impact:

- Theft of session cookies and tokens.
- Execution of arbitrary scripts in the user's browser.
- Potential phishing, defacement, or redirection attacks.

Severity: High

Recommendation:

- Sanitize and validate all client-side inputs before writing to the DOM.
- Avoid unsafe functions like innerHTML or direct DOM manipulation with unsanitized data.
- Implement Content Security Policy (CSP) to block inline scripts.
- Use secure frameworks that auto-escape user input.

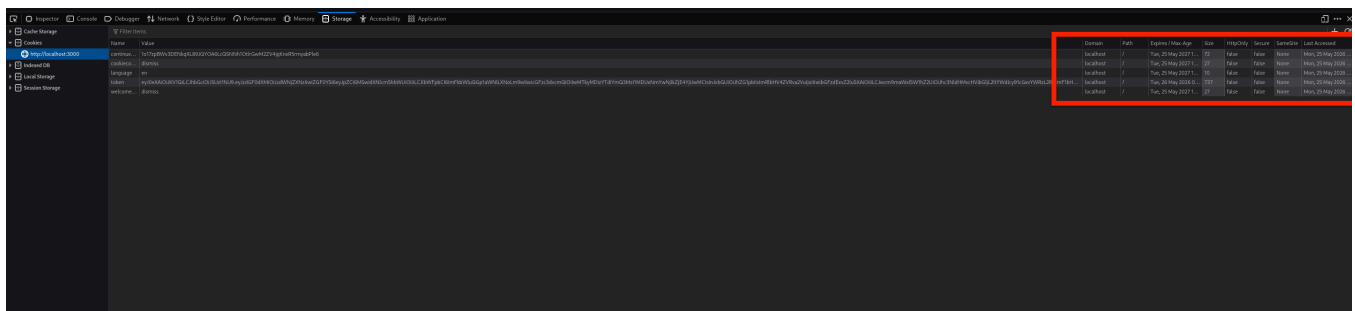
4. Broken Authentication – Insecure Session Cookies

Title: Broken Authentication – Missing HttpOnly Flag on Session Cookies

Description: The application uses session cookies without the HttpOnly attribute. This allows client-side scripts to access authentication tokens, making them vulnerable to theft via Cross-Site Scripting (XSS) attacks. Proper cookie flags are critical for secure session management.

Evidence:

- DevTools inspection revealed session cookies with HttpOnly = false
- Cookies were accessible to JavaScript, increasing the risk of compromise.



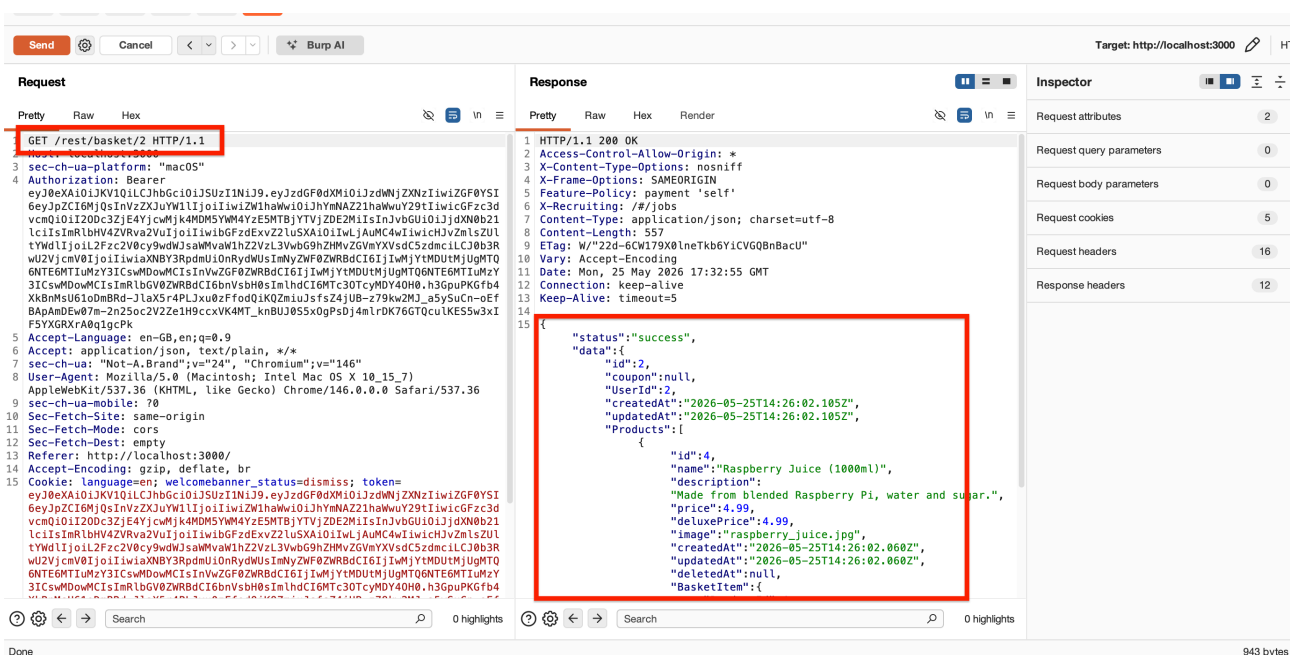
Impact:

- Attackers can steal session cookies using XSS.
- Leads to account takeover and privilege escalation.
- Weakens the overall authentication mechanism.

Severity: High

Recommendation:

- Set HttpOnly flag on all authentication cookies to prevent JavaScript access.
- Use secure flag to ensure cookies are only transmitted over HTTPS.
- Implement session expiration and regeneration after login.
- Adopt strong authentication practices (MFA, token-based auth).



Impact:

- Unauthorized access to sensitive user data.
- Privacy violations and potential account compromise.
- Attackers could view or manipulate other users' shopping activity.

Severity: High

Recommendation:

- Enforce strict server-side authorization checks for every request.
- Replace direct object references with indirect identifiers (UUIDs or random tokens).
- Implement role-based access control (RBAC).
- Conduct regular security testing to detect IDOR vulnerabilities.

Recommendations

• SQL Injection

- Use parameterized queries (prepared statements).
- Apply strict input validation and sanitization.
- Avoid dynamic query construction with string concatenation.

• Cross-Site Scripting (Reflected & DOM-Based)

- Encode and sanitize all user inputs before rendering.
- Avoid unsafe DOM functions (innerHTML, document.write).
- Implement Content Security Policy (CSP).

• Broken Authentication

- Set HttpOnly and Secure flags on cookies.
- Enforce session expiration and regeneration.
- Implement multi-factor authentication for sensitive accounts.

• IDOR

- Enforce server-side authorization checks for every request.
- Replace direct object references with indirect identifiers (UUIDs).
- Apply role-based access control (RBAC).

• General Security Practices

- Regular patching and updates.
- Periodic VAPT cycles to detect new vulnerabilities.
- Developer training on secure coding practices.

Conclusion

The VAPT exercise on the OWASP Juice Shop application revealed **critical and high-severity vulnerabilities** across multiple categories, including injection flaws, cross-site scripting, broken authentication, and insecure object references. The most severe issue **SQL Injection leading to admin account compromise** demonstrates the potential for complete application takeover.

Immediate remediation should prioritize **SQL Injection and XSS vulnerabilities**, followed by strengthening authentication mechanisms and access control. Implementing secure coding practices, enforcing proper cookie flags, and applying server-side authorization will significantly reduce risk.

By addressing these findings, the application will achieve stronger resilience against common web attacks and align more closely with OWASP Top 10 security standards. Regular testing and proactive security measures are recommended to maintain ongoing protection.